

Introduction to Databases

Summative Coursework (cw2)

v25.0

CHANGELOG

v25.0 Initial version.

Database Schema

The schema we use in this assignment models a simplified library scenario consisting of the tables below.

BOOKS have a title (string), an author (string), and genre/subgenre attributes (strings) whose values comply with the following taxonomy:

FICTION includes subgenres

SCI-FI, FANTASY, ADVENTURE, MYSTERY, CRIME, HISTORY, ROMANCE, HORROR

NON-FICTION includes the subgenres

BIOGRAPHY, SELF-HELP, HISTORY, TRAVEL, COOKING, CRIME, REFERENCE

Note that a subgenre may belong to more than one genre; e.g., **CRIME** is a subgenre of **FICTION** as well as of **NON-FICTION**. Each book is uniquely identified by title and author (taken together).

COPIES have a unique identifier (string), a reference (that consists of title and author) to a book, and a location (string).

CUSTOMERS have a unique identifier (string), a name (string), and a branch (string). This table lists all customers (with their ID and name) and the library branch their account is associated with. Note that customers may be associated with a branch that does not stock copies of any book.

LOANS have a unique identifier (string), a reference to a book's copy, a reference to a customer, a start date, and a due date (which does not precede the start date). This table indicates which *copies* are borrowed by which customers, and for how long.

HOLDS have a reference to a customer, a title/author reference to a book, and a queue number (positive integer). The values of the references to customer and book (taken together as a triple) uniquely identify each row (that is, a customer cannot have two distinct holds on the same book). Moreover, the book reference and the queue number (taken together as a triple) form a key for the table as well (that is, two distinct holds for the same book cannot have the same queue number). The queue number is not necessarily serial and provides a total order on the holds for the same book.

RETURNS have a reference to a loan, and a date. Each row is uniquely identified by the loan reference (i.e., there cannot be more than one return for each loan). The date of the return cannot precede the start date of the loan¹ (but can follow its due date, in which case the return is considered *late*).

¹Since current DBMSs do not implement subqueries in **CHECK**, this constraint would require a trigger. Instead, we will simply assume that it holds, even though the schema does not enforce it.

All legal instances of the above schema will not contain any nulls, and *all dates* (in `Loans` and `Returns`) *are assumed to be in the past*. Moreover, with each loan we associate a discrete interval of dates, which is

- $[s, e]$ if the loan has a return (i.e., a corresponding row in the `Returns` table), and
- $[s, +\infty)$ otherwise,

where s is the loan's start date, and e is its return date. Then, we assume that *distinct loans of the same copy cannot have overlapping intervals* (i.e., the intervals have no date in common).

SQL schema. The detailed SQL schema can be found on Learn, as well as on DICE's file system (AFS) at :
</afs/inf.ed.ac.uk/group/teaching/dbs/2025/cw/c2/sql/schema.sql>

REAL schema. In REAL, the schema is as follows:

```
Books (title, author, genre, subgenre)
Copies (copy_id, title, author, branch)
Customers (cust_id, name, branch)
Loans (loan_id, copy, customer, start, due)
Holds (customer, title, author, queue)
Returns (loan, date)
```

There are some differences with respect to the SQL schema:

- Attributes are not ordered, and relation and attribute names are case-sensitive. Note that, in the above schema, relation names start with a capital letter, whereas attribute names are in lower case.
- Constraints are not enforced, but you **must** assume they are satisfied (so, e.g., there cannot be two records in `Returns` with the same value for `loan` and different values for `date`, even though REAL, which does not implement constraints, will not complain if you create an illegal CSV instance violating this assumption).
- The values of all dates (i.e., `start` and `due` in `Loans`, and `date` in `Returns`) are expressed as numbers in the *Julian Date* system.

Relational Algebra

Write the following 5 queries in Relational Algebra *under bag semantics*.

(1) **Customers who never borrowed books from their own branch**

Return the ID and branch of each such customer, without repetitions. Customers who have never borrowed a book (from any branch) will not be in the output.

The output table will have two columns: one named `customer` and consisting of distinct customers' IDs, the other named `branch` and consisting of library branches.

(2) **Customers who always return their loans on time**

Return the ID and name of each such customer, without repetitions. Customers who have never borrowed a book will not be in the output. Observe that loans for which there is no return are automatically late (because due dates are assumed to be in the past; see remark in the schema specification above).

The output table will have two columns, called `cust_id` and `name`, consisting of customers' IDs and names, respectively.

(3) **Books for which the first hold in the queue can be fulfilled locally**

These are books of which at least one copy is available and the hold with the lowest queue number was placed by a customer whose account is in any of the branches with available copies of that book. Return the title and author of each such book, and the ID and branch of the customer with the corresponding hold. Books on which there are no holds will not be in the output. Each answer record occurs only once.

The output table will have four columns: `title` and `author`, which together identify a book, and `cust_id` and `branch`, consisting of customers' IDs and branches, respectively.

(4) **Books for which there is at least one available copy in each branch that stocks copies of that book**

In other words, books such that each branch stocking at least one copy of the book has at least one of those copies available to borrow. Return the title and author of each book that satisfies this requirement, along with each branch in which there is an available copy of that book. Each answer record occurs only once.

The output table will have three columns: `title` and `author`, which together identify a book, and `branch`, consisting of library branches.

(5) **Customers who borrowed the same book more than once**

Return the ID of each such customer, the title and author of *each* book they borrowed multiple times, and the start date of the last loan (of that book by that customer).

The output table will have four columns: `customer` consisting of customers' IDs, `title` and `author` consisting of books' titles and authors, respectively, and `last_loan` and consisting of dates. No two rows in the output will have the same values for `customer`, `title`, and `author`.

Requirements

- Queries must be written in the Relational Algebra syntax accepted by REAL v0.7 or above.
- Each query must be written in a plain-text file named `ra_N.txt` (all lowercase) where N is the number that identifies the query in the above list. For example, the second query ("Customers who always return their loans on time") will be written in the file `ra_2.txt`.
- The character encoding of the files must be ASCII, or UTF-8 without BOM. It is *strongly* advised to prepare the queries on DICE using the `nano` text editor or similar (e.g., `emacs`, `vim`), and to check the encoding with the `file` command.

- Each file must contain a single Relational Algebra expression, possibly spanning multiple lines and *optionally* terminated by semicolon. REAL's commands (starting with a dot) and view definitions (specified with `:=`) are not allowed.
- The order of columns in the output is irrelevant (there is no order), but their names are important (recall also that they are *case-sensitive*) and must match exactly those given in the query specification. The order in which the rows appear in the answers does not matter.

SQL

Write the following 5 queries in SQL.

(1) Number of loans and their average length, per stocked book

For each book of which there is at least one copy in stock², compute how many times that book was borrowed and the average number of days, from when the loan starts until it is returned, it was borrowed for. The average must be *formatted as a number with exactly 1 decimal place*, and active (i.e., not yet returned) loans do not contribute to it. Books for which each of their copies was either never borrowed or is on active loan will appear in the output with `NULL` average³. Note that subtracting two dates yields the number of days between them; this quantity is positive or negative depending on which date is greater, and it is zero when the dates are equal. Each answer row occurs only once.

The output table will have four columns: the first and the second consist of books' titles and authors, respectively; the third is the number of loans, and the fourth is the average length of loans (formatted as required).

(2) Number of active loans, on-time returns, and late returns per customer

For each customer, return their ID, followed by the number of loans the customer has not yet returned, the number of those returned no later than their due date, and the number of those returned late. Customers who do not have any loans (neither active nor returned) appear in the output with all the above quantities set to 0. Each answer row occurs only once.

The output table will have four columns: the first consists of customers' IDs, while the second (active loans), third (on-time returns), and fourth (late returns) consist of non-negative integers.

(3) Number of stocked books and customers in each branch

For each branch, compute the total number of distinct books for which that branch has copies in stock (irrespective of whether they are available to borrow or not), and the number of customers who have an account in that branch. Branches that do not stock (copies of) any book will be in the output with 0 books, and branches without customers will be in the output with 0 customers. Each answer row occurs only once.

The output table will have three columns: the first one consists of distinct branches; the second (book count) and third (customer count) consist of non-negative integers.

(4) Number of borrowers and copies for each category of book

Let us refer to any legal combination of genre/subgenre values as a *category*. For each distinct category present in the `Books` table, compute (a) how many distinct customers borrowed at least one book in that category, and (b) how many copies of books in that category are in stock. The status of a loan (active/returned) is irrelevant, that is, qualifying loans contribute to the computation of (a) irrespective of whether they have been returned or not. Similarly, the availability of a copy is irrelevant for the computation of (b), that is, all copies of a book in a given category contribute to the total for that category, independently of whether they are available to borrow or not. Each answer row occurs only once.

The output table will have four columns: the first consists of genres, the second of subgenres, while the third and the fourth contain non-negative integers resulting from the computation of (a) and (b) above, respectively.

(5) Customers who always return all of their previous loans before starting a new one

These are customers who, on each date in which they take out a loan, have returned all of their previous loans (i.e., those started on an *earlier* date) before or on the date in which the new loan starts. For each such customer, return their ID and the most recent date in which they started a loan. Customers with only one loan will trivially be in the answer, but customers with no loans at all will not.

The output table will have two columns: the first one consists of distinct customers' IDs, the second of dates.

²This means that there is a branch with a copy, irrespective of whether it is available to borrow.

³Books for which some copies have never been borrowed and all others are on active loans fall into this category; so do books for which all of their copies have never been borrowed, and those for which all of their copies are on active loans. Note also that you cannot directly use `NULL` in your query, nor `LEFT/RIGHT` joins.

Requirements

- Each query must be written in a plain-text file named `sql_N.txt` (all lowercase) where N is the number that identifies the query in the above list. For example, the first query (“Number of loans and their average length, per stocked book”) will be written in the file `sql_1.txt`, and so on. The character encoding of the files must be ASCII, or UTF-8 without BOM. As for Relational Algebra, it is advisable to prepare the queries on a DICE machine.
- Each file must contain a single SQL query statement, possibly spanning multiple lines, *without comments*, and optionally terminated by semicolon. Only the standard SQL constructs we have seen in class are allowed, , with the exception of those requiring forbidden words (see below).
- Moreover, the queries cannot use any of the words listed in the file `/afs/inf.ed.ac.uk/group/teaching/dbs/2025/cw/c2/sql/forbidden.txt` (this is also available on Learn), *not even as identifiers* (that is, column/table/-subquery aliases).
- The names of the columns in the output table are not important (as long as they are not among the forbidden words). What matters is the number of columns, their type, and which position they appear in, which must match the order indicated in the query specification.
- The order in which the rows appear in the output is irrelevant for this assignment. No ordering is enforced on the answers, so the DBMS will output rows in an arbitrary order.